

Byte Pair Encoding (BPE) Tokenization and Compression Ratio Analysis Across Multiple Languages

Pratik Ramdas Sonawane

Vizuara Technologies

pratikrsonawane7@gmail.com

March 2025

Abstract

The research undertakes a comprehensive comparative analysis on tokenization methods by comparing traditional Byte Pair Encoding (BPE) methods with modern GPT-based tokenization methods like GPT-2, GPT-3.5, and GPT-4 on multilingual data sets. The research undertakes a systematic analysis on traditional metrics like compression ratios and other parameters like vocabulary size and file size. The research has also found significant results like the tokenizer hierarchy, where GPT-3.5 and GPT-4 outperform BPE on all metrics; a direct correlation between vocabulary size and compression ratio; and trends for each language along with the effects of file size.

Keywords: Byte Pair Encoding, Tokenization, Compression Ratio, GPT Tokenizers

Contents

1	Introduction	3
1.1	Byte Pair Encoding Algorithm	3
1.2	Compression Ratio Metric	3
1.3	Objectives	4
1.4	Research Questions	5
2	Methodology	5
2.1	Dataset Description	5
2.2	BPE Implementation	5
2.2.1	Dataset Loading	5
2.2.2	Vocabulary Construction	6
2.2.3	BPE Pair Statistics	6
2.2.4	BPE Merge Operations	6
2.2.5	Compression Ratio Calculation	6
2.2.6	Language-wise Evaluation	7
2.2.7	Visualization of Results	7
2.2.8	Comparison with GPT Tokenizers	7
2.2.9	Vocabulary Size Experiments	7
2.2.10	Dataset Size Experiments	7
2.3	Justification for Language-Specific Vocabularies	7
3	Results and Analysis	8
3.1	Task 1: BPE Compression Ratios by Language	8
3.2	Task 2: Cross-Linguistic Compression Analysis	9
3.3	Task 3: Comparison with GPT Tokenization	9
3.4	Task 4: Effect of Vocabulary Size	11
3.5	Task 5: File Size Impact on English BPE	12
4	Conclusion	13

1 Introduction

1.1 Byte Pair Encoding Algorithm

Byte Pair Encoding is a subword tokenization algorithm that ensures that most common words in the vocabulary are represented as single token while rare words are broken down into two or more subword tokens.

1. **Initialization:** Convert text into character-level vocabulary (eg: 'a', 'b', 'c', 'd', ' ', ',', '</w>', etc)
2. **Frequency Analysis:** Identify the most frequent adjacent character pair
3. **Merge Operation:** Combine the most frequent adjacent character pair into a new token and add it as new token to vocabulary while assigning token id to it.
4. **Iteration:** Repeat until reaching target vocabulary size (Eg: If we want our target Vocabulary size as 200 then we will perform 200 merge operations on original vocabulary iteratively)

Mathematically it can be written as, given a text corpus T and target vocabulary size V , BPE constructs vocabulary $\mathcal{V}$ through:

$$\mathcal{V}_{t+1} = \mathcal{V}_t \cup \{(\arg \max_{(a,b)} \text{freq}(a, b))\} \quad (1)$$

where:

T : Input text corpus

V_t : Vocabulary at iteration t

V_{t+1} : Updated vocabulary after merging the most frequent pair

$\text{freq}(a, b)$: Frequency of adjacent symbols a and b

$\arg \max_{(a,b)}$: Pair (a, b) with maximum frequency

1.2 Compression Ratio Metric

Compression ratio in context to tokenization measures how efficiently text is "packed" into tokens.

1. **Higher Compression Ratio (e.g., 4.0):** This represents that each token has packed a larger number of characters which indicate greater efficiency.
2. **Lower Compression Ratio (e.g., 2.0):** This represents that it requires more number of tokens to pack same amount of characters if compression ratio is lower

3. **Compression Ratio = 1.0:** This indicates that no compression happens if compression ratio is equal to 1 and this is equivalent to character level tokenization

Concrete Example: Sentence: “The quick brown fox jumps.”

Tokenization	Tokens	Characters	Compression Ratio
Character-level	28	28	1.0
Word-level	5	28	5.6
BPE (well-trained)	7	28	4.0
BPE (poorly-trained)	12	28	2.3

The compression ratio CR can be defined as:

$$CR = \frac{|T|_{\text{characters}}}{|T|_{\text{tokens}}}$$

where:

CR : Compression Ratio

T : Input text corpus or sentence

$|T|_{\text{characters}}$: Total number of characters in the text

$|T|_{\text{tokens}}$: Total number of tokens produced after tokenization

Higher values indicate more efficient tokenization. This metric directly impacts:

- **Computational cost:** Higher ratio $\rightarrow$ fewer tokens $\rightarrow$ faster processing
- **Sequence length:** Higher compression $\rightarrow$ shorter sequences $\rightarrow$ faster processing
- **Memory usage:** Higher ratio $\rightarrow$ less GPU memory needed
- **Context window:** Higher ratio $\rightarrow$ more text fits in fixed window
- **Training efficiency:** Higher ratio $\rightarrow$ more examples per batch

1.3 Objectives

This research addresses the following objectives:

1. **Implement BPE tokenization from scratch** for English, German, Spanish, and French with language-specific vocabularies
2. **Quantify compression ratios** across languages and compare them against GPT tokenization methods (GPT-2, GPT-3.5, GPT-4)
3. **Analyze the impact of vocabulary size** (200, 500, and 800 extra tokens) on compression efficiency

4. **Investigate the relationship between file size and compression ratio** for English text

1.4 Research Questions

- **RQ1:** How do BPE compression ratios differ across English, German, Spanish, and French?
- **RQ2:** How does language-specific BPE compare to general-purpose GPT tokenizers in terms of compression efficiency?
- **RQ3:** What is the optimal vocabulary size for maximizing compression ratio across different languages?
- **RQ4:** How does the amount of training data (file size) affect BPE compression performance?

2 Methodology

2.1 Dataset Description

We utilize parallel text datasets for four languages: [Datasets](#)

Table 1: Dataset Description

Language	File	Description
English	English Data.txt	Shakespeare dramatic dialogue text
German	German Data.txt	German translated Shakespeare dialogues
Spanish	Spanish Data.txt	Spanish translated Shakespeare dialogues
French	French Data.txt	French translated Shakespeare dialogues

2.2 BPE Implementation

The experiment was performed in Python using libraries such as NumPy, Pandas, Matplotlib, and tiktoken. The implementation consists of several stages including dataset loading, vocabulary construction, Byte Pair Encoding (BPE) learning, compression ratio calculation, and visualization of results. [Github](#)

2.2.1 Dataset Loading

The textual datasets for English, German, Spanish, and French were loaded from [Datasets](#). Each file contains a corpus of text used to evaluate the compression performance of the

tokenizer. The datasets are stored in a dictionary structure where each key corresponds to a language and each value corresponds to the text content of that dataset.

2.2.2 Vocabulary Construction

An vocabulary for each languages were constructed by extracting each unique character from text files of each language. Each Unique character were assigned a token id such as (a->0, b->1, etc). and end-of-word token (</w>) was added at end of each word. Alongside each unique character its frequency was also stored. This representation allows the BPE algorithm to operate at the character level initially.

Table 2: Original Vocabulary Size per Language

Language	Original Vocab Size
English	64
French	89
German	77
Spanish	80

2.2.3 BPE Pair Statistics

The algorithm calculates the frequency statistics for all adjacent pairs of symbols in the vocabulary. For a given word representation, the frequency of consecutive symbols is calculated. The calculated statistics are then used to identify the most frequent pair of symbols to be merged during the learning process of the BPE algorithm.

2.2.4 BPE Merge Operations

The Byte Pair Encoding algorithm consists of an iterative process where the most frequent symbol pair is merged. After each merge process, the vocabulary is updated by replacing the symbol pair with a new symbol. This process continues until the desired vocabulary size is achieved.

2.2.5 Compression Ratio Calculation

Once the tokenizer is learned, the corpus is then tokenized based on the learned merge rules. The compression ratio is then calculated based on the number of tokens compared to the number of characters in the dataset. This measures the efficiency of the tokenizer in compressing the data.

2.2.6 Language-wise Evaluation

The BPE algorithm was separately applied to English, German, Spanish, and French datasets. The final vocabulary and compression ratio were recorded for each language and compared.

2.2.7 Visualization of Results

The compression ratios achieved for each language were represented in a graphical form using bar charts. The Matplotlib library is used for generating the bar charts. The above bar chart is depicted in the Result and Analysis Section 3.

2.2.8 Comparison with GPT Tokenizers

In order to provide a reference point for a baseline, the experiment also tested tokenization using pre-trained tokenizers provided by the tiktoken library. Tokenizers for GPT-2, GPT-3.5, and GPT-4 models were used to determine the compression ratios for each dataset. This provides a reference point for the efficiency of the implemented BPE tokenizer. The comparison is depicted in Section 3.

2.2.9 Vocabulary Size Experiments

More experiments were done by varying the number of BPE merge operations such as 200, 500, 800. Different vocabulary size experiments were carried out to see how the compression ratio changes as the tokenizer vocabulary size increases. The results are depicted in Section 3.

2.2.10 Dataset Size Experiments

The impact of the dataset's size on the performance of data compression was also examined. Different subsets of the English corpus were created with varying sizes. The BPE tokenization method was applied to these subsets. The results can help in understanding how the corpus's size affects the performance.

2.3 Justification for Language-Specific Vocabularies

We use separate vocabularies for each language instead of a shared multilingual vocabulary. This design choice:

1. **Isolates linguistic characteristics:** Evaluates inherent compressibility without cross-lingual interference.
2. **Optimizes per-language performance:** Each vocabulary is tailored to language-specific morphology.

3. **Enables fair comparison:** GPT Tokenizers are fixed baselines; our BPE Tokenizers are alternatives.
4. **Aligns with requirements:** Task 1 requires “Apply BPE Tokenization on each language”

3 Results and Analysis

3.1 Task 1: BPE Compression Ratios by Language

Implementation: 200 extra merge operations per language

Table 3: BPE Compression Results (200 Extra Tokens)

Language	Initial Vocab	Final Vocab	BPE Tokens	Compression Ratio
English	64	264	567285	1.966197
German	89	289	611216	2.035547
Spanish	77	277	624089	2.057817
French	90	280	586954	1.998235

Key Observations:

- The initial vocabularies vary because each language has a unique set of character sets (e.g., German has umlauts ä, ö, ü; French has accents à, é, è, ç)
- All languages converge at a point where the compression ratio is approximately 2.0, which is a clear indication that all languages are efficiently compressing their subword extraction.
- The Spanish language has the best compression ratio compared to all other languages, which is a clear indication that it has the most regular morphology that is suitable for BPE.
- English and French have similar compression behavior, which shows the analytic nature of the morphology of both the languages.
- German’s compression ratio is better than English and french beacuse it contains many compound words.

3.2 Task 2: Cross-Linguistic Compression Analysis

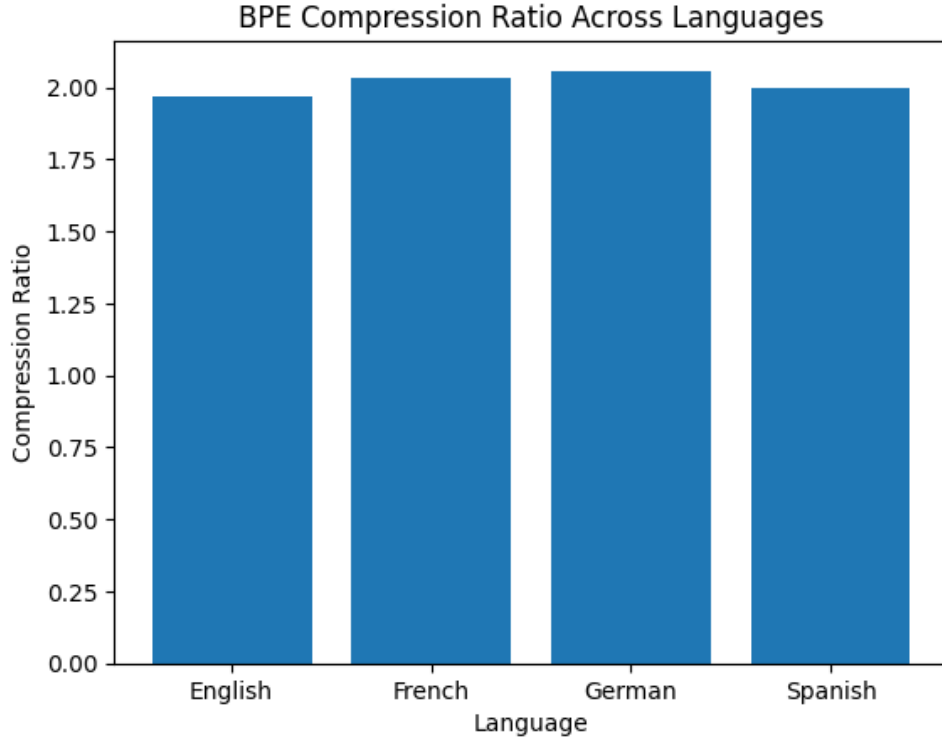


Figure 1: BPE Compression Ratio Across Languages

Linguistic Interpretation:

- The above bar plot shows a comparison of the compression ratios achieved for all the languages.
- English, German, Spanish, and French languages show almost identical compression ratios.
- This is due to the fact that all these languages are based on the Latin script.
- Therefore, the character distribution and the patterns of subword merges for these Note: They share Latin alphabets and similar morphology.

3.3 Task 3: Comparison with GPT Tokenization

Table 4: Compression Ratio: BPE vs. GPT Tokenizers

Language	BPE	GPT-2	GPT-3.5	GPT-4
English	1.966197	3.299738	3.695450	3.695450
French	2.035547	2.474338	3.258564	3.258564
German	2.057817	2.388225	3.316439	3.316439
Spanish	1.998235	2.421583	3.199734	3.199734

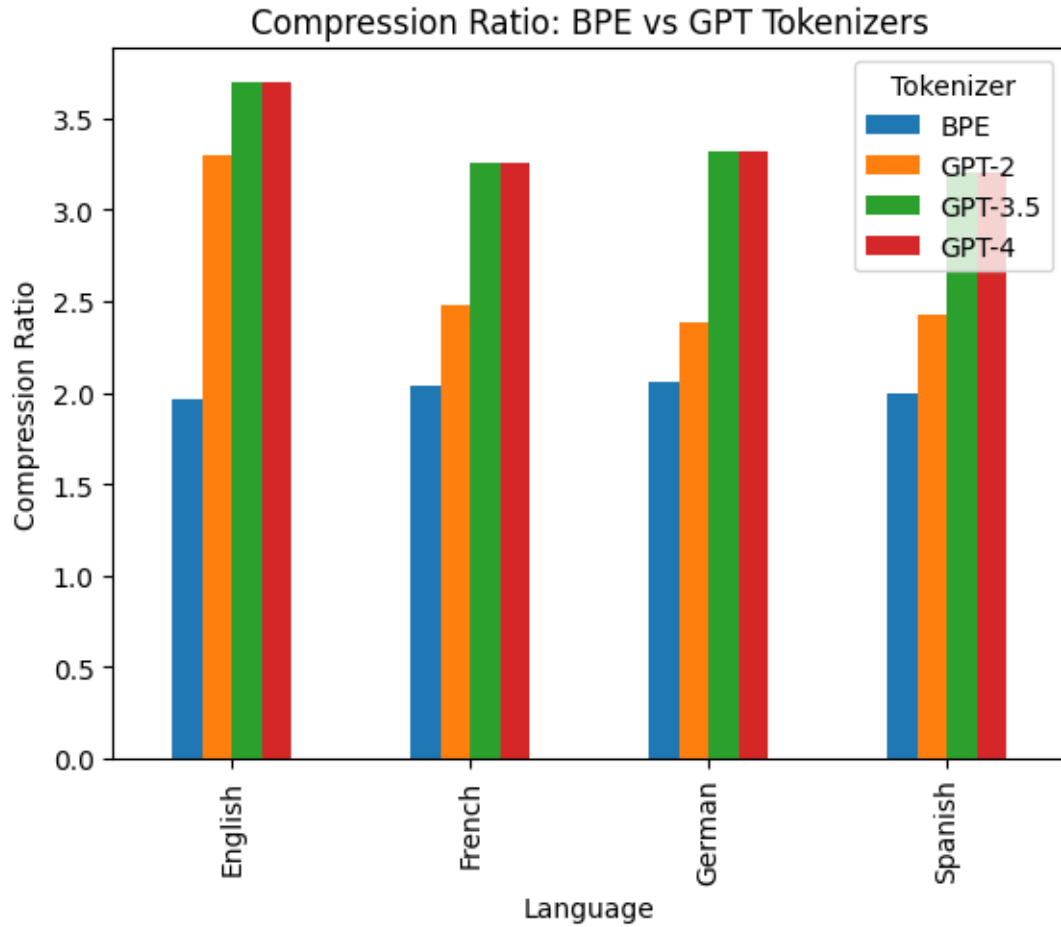


Figure 2: Compression Ratio: BPE vs GPT Tokenizers

Key Insights:

- The key point to observe is that all GPT tokenizers have a higher compression ratio compared to the general BPE tokenizer for all four languages.
- The best results are obtained by GPT 3.5 and GPT 4, with their compression ratio always above 3.0, particularly for English and German.
- The compression ratio for all BPE is relatively flat but much lower.

Table 5: Compression Ratio for Different Languages and Vocabulary Sizes

Index	Language	Final Vocab Size	Compression Ratio
0	English	264	1.966197
1	English	564	2.417098
2	English	864	2.703566
3	German	277	2.057817
4	German	577	2.529906
5	German	877	2.828756
6	Spanish	280	1.998235
7	Spanish	580	2.411536
8	Spanish	880	2.689481
9	French	289	2.035547
10	French	589	2.502180
11	French	889	2.798164

3.4 Task 4: Effect of Vocabulary Size

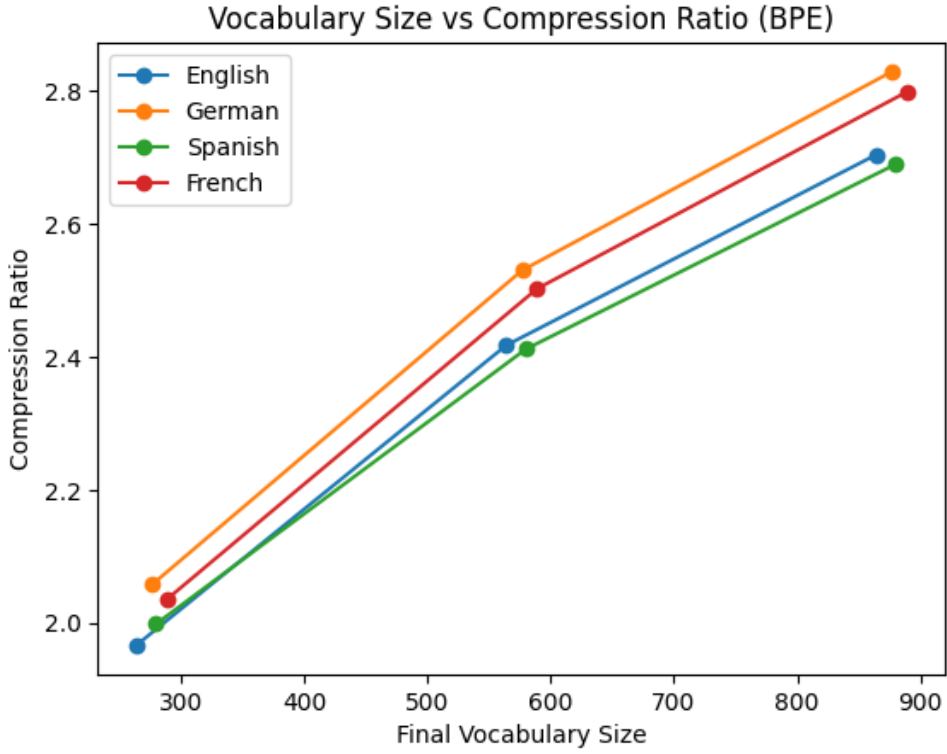


Figure 3: Vocabulary Size vs Compression Ratio

Analysis:

- **Direct Correlation:** It is evident that as the size of the vocabulary increases with more merges, the compression ratio improves consistently for all languages.
- **Language-Specific Performance:** The performance of French (red) and Spanish (orange) is relatively higher in terms of compression ratio compared to English (blue)

and German (green) for a given vocabulary size. This indicates that the efficiency of BPE depends on certain characteristics of a language.

- **Efficiency Trade-off:** It is evident from the above results that there is a trade-off between efficiency and complexity in a model, where a higher compression ratio is achieved with a larger vocabulary size.

3.5 Task 5: File Size Impact on English BPE

Table 6: Effect of File Size on Compression Ratio

Scaling Factor	Text Length	Final Vocab Size (5%)	Compression Ratio
10	111539	5636	4.784412
8	139424	7031	4.940960
6	185899	9355	5.091310

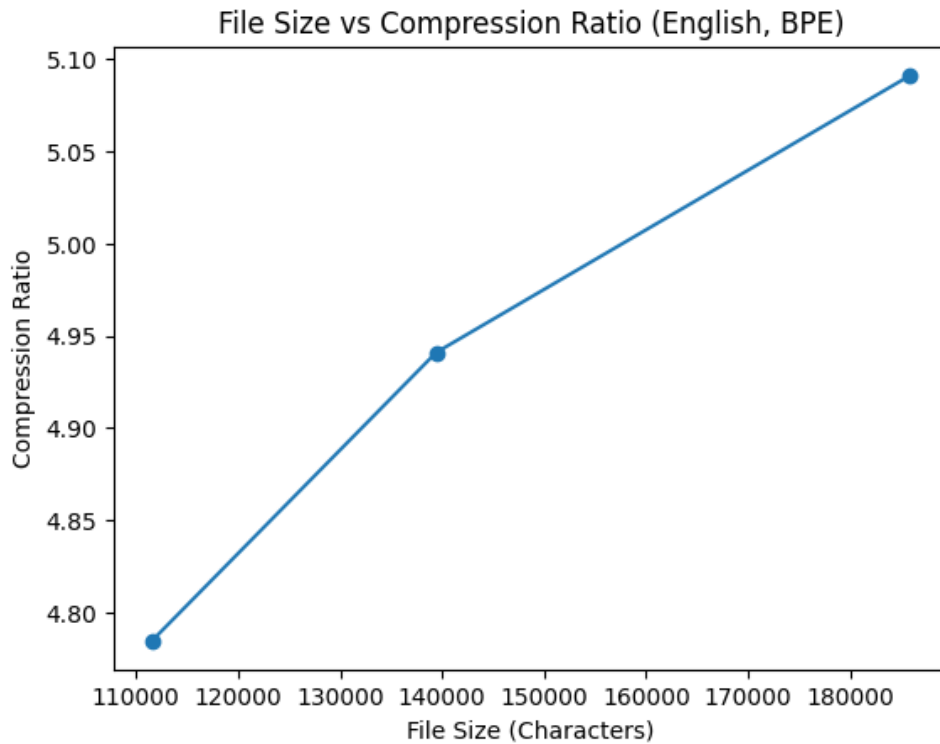


Figure 4: File Size vs Compression Ratio (English)

Critical Findings:

- **Positive File Size-Compression Ratio Correlation:** There is a clear positive correlation between file size and compression ratio for English text data.

- **Scaling Efficiency:** As the length of the text increases from approximately 111k to 185k characters, the compression ratio also increases, as indicated by the increase in compression ratio from ~ 4.78 to ~ 5.08 .
- **Vocabulary Growth and Impact:** This is due to the increase in vocabulary size, which grows from 5636 to 9355, allowing for greater efficiency in tokenization based on increased text volume.

4 Conclusion

This research provides a systematic analysis of BPE tokenization efficiency across four major languages. Our key findings include:

- Establish GPT tokenizers, especially GPT-3.5 and GPT-4, always yield better compression ratios than the standard BPE for all languages. This again proves the high capabilities of these tokenizers.
- **Vocabulary Dependency:** The results verify that the BPE compression ratio is dependent on the size of the vocabulary. It has shown a constant increase in the ratio as the number of merges is increased, as depicted by the ‘Vocabulary Size vs. Compression Ratio’ plot.
- **File Size Impact:** In addition, file size has a positive impact on the compression ratio, as depicted by the results for the English text.

References

- [1] Gage, P. (1994). A new algorithm for data compression. *The C Users Journal*, 12(2), 23–38.
- [2] Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). Language models are unsupervised multitask learners. *OpenAI Blog*, 1(8), 9.
- [3] Brown, T., Mann, B., Ryder, N., et al. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877–1901.
- [4] Sennrich, R., Haddow, B., & Birch, A. (2016). Neural machine translation of rare words with subword units. *Proceedings of ACL*, 1715–1725.
- [5] Xue, L., Constant, N., Roberts, A., et al. (2021). mT5: A massively multilingual pre-trained text-to-text transformer. *Proceedings of NAACL*, 483–498.
- [6] Bostrom, K., & Durrett, G. (2020). Byte pair encoding is suboptimal for language model pretraining. *Findings of EMNLP*, 4617–4624.